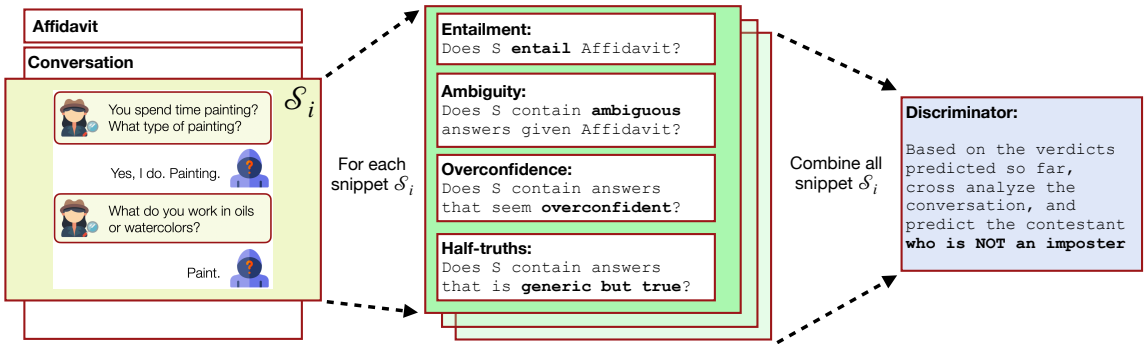

Figure 2: Pipeline of the bottleneck model deriving bottleneck controls and the discriminator collates them for final prediction. We use few-shot LLMs to extract such controls that outperform an end-to-end approach.

compensation for the contestants (imposters), leading to uncertainties regarding the participants' true intent to deceive.

3.3 The T4TEXT Dataset

Each data point in T4TEXT entails three main components from an independent game session: the name of the real CC, the affidavit containing the objective truth about the CC, and the conversations (Q/As) between the judges and the contestants. Every data point, on average, has 12-15 Q/A pairs.

Our dataset is novel since it is based on conversations around the 1950s before social media and the internet existed. Unique interpersonal interactions exist in T4TEXT, for instance, Edmund Hillary, the first person to summit Mount Everest in 1953, was one of the CCs in a session, but his appearance was unknown within the US entertainment industry, analogous to contemporary online crimes with unknown scammers.

Ambiguity/Randomness. According to Ekman (1997), liars cannot keep their claims consistent, leading to ambiguity that exposes their lies. Figure 1 shows an example where the contestant mentions Pennsylvania while Tennessee is being discussed; this may have been an oversight on the contestant's part, or it may be a random interjection to stall a conversation; either way, it highlights the possibility that the contestant could be a liar.

Overconfidence. Overconfidence in the context of deception has been characterized in three ways: overestimating one's actual performance, overplacing one's performance in comparison to others, and excessive precision in one's beliefs (Moore and Healy, 2008). This behavior is similar to the contestants' intent to mislead the judges in Figure 1 where the contestant lied too confidently (Serra-Garcia and Gneezy, 2021) and hence made a small but important factual error for judges to understand that the individual is an imposter.

Half-truths. In deceitful conversations, half-truths are less sinful than outright lies and explicit distortions (Carson, 2010). Given the constraint of the real CC to answer truthfully, uttering half-truths becomes a prominent strategy to suppress facts vital for identification (DePaulo et al., 2003). Figure 1 illustrates a contestant who does not mention what paint they use even after repeated questioning.

Dataset statistics. T4TEXT is comprised of 150 data points with a volume of 86,746 words. There are 1546 utterances, including both judges and contestants as speakers. 450 unique contestants appeared in 150 sessions (datapoints), but judges reappeared from a unique set of size 56.

4 Approach

We investigate the possibility of using an LLM to uncover informative cues from the language of deception. Here we define the task, our base models, and a bottleneck model capable of reasoning the language cues for successful deception detection.

4.1 Task

We define the truth detection task as follows: given the name, affidavit ($\mathcal{A}$), and a conversation ($\mathcal{C}$) as input x, predict the real contestant as y from an output label space of `Number One`, `Number Two`, `Number Three`. This is a discriminative task set up in the form of a 3-way classification problem. We use the terms *deception detection* and *truth identification* interchangeably in the rest of the paper.

4.2 Base Models

Our base model is an LLM. We initiate the base model with a **task prompt** that includes the brief

description and rules of T4TEXT such as all contestants are incentivized to lie with the constraint that real CC still has to stick to the truth. The input followed by the task prompt contains the name of the CC, the affidavit, and the conversation between the judges and the contestants. Please refer to the appendix for the complete prompt.

4.3 Bottleneck Models

Our base models process the input end-to-end to predict the real CC by performing necessary reasoning implicitly. However, implicit reasoning cannot be attributed to literature-backed linguistic cues, as discussed in Section 3.3. These cues can be explicitly extracted and used as a features. However, to ensure models restrict reasoning with these features, we use bottleneck models (Koh et al., 2020).

Identifying real CC involves assessing Q/A pairs addressed to a contestant at a time, as a **snippet** ($\mathcal{S}$) of the conversation and assessing the likelihood of the addressed contestant being the real CC. Our bottleneck models are employed through a set of bottleneck controls, which are the high-level predictors for deception detection (from Section 3.3).

A bottleneck model takes the form of $f(g(\mathcal{S}))$; where g is a mapping function that maps the input snippet $\mathcal{S}$ to a bottleneck control, predictive for deception and f is the final discriminator that maps the intermediate bottleneck controls to the output label space. The success of the final prediction depends on the success of the intermediate functions generating bottleneck controls. We use LLMs for both f and g. For each g, we write a bottleneck prompt ($p_{\text{bottleneck}}$) for each control, which we discuss here (also see Figure 2). This simulates the systematic uncovering of these cues over the conversation, as the original judges would do.

Bottleneck controls.

- **Entailment:** As per game rules, each answer from the contestants should be verified in the light of the affidavit. We view this as an entailment task (Tafjord et al., 2022). Given a premise and a hypothesis, an entailment task would be to predict if the hypothesis entails, contradicts, or does not relate to the premise. We set the affidavit $\mathcal{A}$ as the premise and a snippet $\mathcal{S}$ as the hypothesis and predict one of these: entail, contradiction, or neutral.

- **Ambiguity/Randomness:** Each snippet contains ambiguous or deliberately random responses from the contestants, indicative of deception (see Figure 1). We develop a bottleneck prompt that takes a snippet $\mathcal{S}$ as an input to predict control values: ambiguous or unambiguous, in the light of the contestant being deceptive.

- **Overconfidence:** Similarly as above, the next bottleneck prompt ascertains if the responses reveal overconfidence in a contestant (e.g., Figure 1), indicating deception. The model predicts a verdict: overconfident or neutral.

- **Half-truths:** Finally, we develop a bottleneck prompt to decode an utterance as a half-truth (example in Figure 1) to predict if the snippet contains half-truths and hence is indicative of deception or not.

While the bottleneck controls are predicted for each conversation snippet, they can be derived either *independently* or *sequentially*. It is analogous to the original setting, where the snippets appear one by one, with the possibility that an older snippet may influence future questions from the judges and future answers from the contestants. For independent bottleneck controls, the mapping function is realized as $g(\mathcal{S}_i)$ for the i-th snippet. For sequential bottleneck control, the mapping function takes the form $g(\mathcal{S}_1, \cdots, \mathcal{S}_{i-1}, \mathcal{S}_i)$.

Discriminator. The discriminator function $f(\cdot)$ takes annotated part of the conversations with the derived bottleneck controls for every snippet, to predict the real CC.

Hyperparameters. We use OpenAI LLMs as the candidate base models and also for f and g: `text-davinci-003` (Brown et al., 2020), `gpt3.5-turbo-16k`, and `gpt4` (OpenAI, 2023). For all GPT-3.5/4 experiments, we used `temperature` = 0, `max_tokens` = 1024, and `top-p` = 1 (for nucleus sampling). The system prompt, the user prompt (for base models), and the bottleneck prompt are provided in Figures 5 and 6. We experimented with temperatures of 0.2 and 0.7, and the difference in the results was not statistically significant. Similarly, `top-p` = 0.95 did not yield any statistically significant different results.

4.4 Baselines and Evaluation

One of the primary baselines for our system is to compare the model's performance with human performance.

Base models. For zero-shot models, our primary baselines will be the base models with all LLM variants that do not break the decision-making process through bottlenecks. Kojima et al. (2022) show encouraging performance when a chain-of-thought (CoT) prompt is added to a zero-shot LLM: "`Let's think step by step`,"—becomes our baseline.

Supervised Models. For completeness, we also consider three supervised baselines where we only train the discriminator f using XGBoost classifier, mirroring (Soldner et al., 2019). For the features required for the XGBoost classifier, we consider two options: psycholinguistic features from Soldner et al. (2019) and `gpt-3` embeddings of derived bottleneck controls from our `gpt-4`-based bottleneck model. For the LIWC-supervised baseline, we generate LIWC features for responses given by each contestant and concatenate them for the complete feature vector for the classifier. Finally, we train a BERT model (Devlin et al., 2019). For a fair comparison, we evaluate supervised models by a leave-one-out scheme spanning the full T4TEXT dataset.

Bottleneck Models. For variations of our bottleneck approach, we create all possible combinations for f and g with our LLM variants. `gpt-4` as both $f(\cdot)$ and $f(\cdot)$ is our model, and rest 8 are baselines. We ablate four bottleneck controls individually while keeping the rest the same to compare with our model. We also evaluate if independent or sequential bottleneck derivation affects model performance. Finally, Soldner et al. (2019) suggests LIWC features (Ott et al., 2013) are effective in predicting hidden intents in deceptive communication. We use such LIWC features as $g(\cdot)$, an alternative to our bottleneck features, pairing them with a `gpt-4` based discriminator. For all baselines, prompts are provided in the appendix.

Evaluation To evaluate model performance, we use accuracy and accuracy@2; the latter denotes if the correct prediction appears in the top two guesses. We use the session-level macro-average accuracy for human performance, as every session (one datapoint in T4TEXT) has predictions from 4 judges. We use a pairwise comparison in AMT and an absolute metric to evaluate the quality of the generated explanations from the models. For pairwise comparison, we measure % of times explanations generated by our model are preferred by 3 human evaluators (in the majority) than explanations from a competing baseline. Following Majumder et al. (2021), we use the e-ViL score on explanation where the models predicted accurately. We ask the annotators *if an explanation is satisfactory* with four options: yes, partial-yes, partial-no, and no. This required us to take an intersection of samples when both comparing models generated correct predictions, an average of which was 31.

Models	**Acc** (↑)	**Acc@2** (↑)	**% wins** (↑)	κ (↑)
Human*	41.3	–	–	–
Random	33.3	66.6	–	–
Base Models (best setups)				
GPT-3, 2-shot	29.3	56.0	100	0.85
GPT-3.5, 2-shot	33.3	70.0	97	0.81
GPT-4, 2-shot	34.7	72.0	90	0.74
CoT Models (best setups)				
GPT-3, 2-shot	27.3	55.3	100	0.91
GPT-3.5, 2-shot	30.0	65.3	100	0.91
GPT-4, 2-shot	32.0	64.7	97	0.84
Bottleneck Models (best combinations)				
f: GPT-3, g: GPT-4, 0-shot	29.3	57.3	93	0.84
f: GPT-3, g: GPT-4, 2-shot	30.0	58.1	93	0.85
f: GPT-3.5, g: GPT-4, 0-shot	35.3	71.3	77	0.70
f: GPT-3.5, g: GPT-4, 2-shot	36.0	72.0	77	0.73
f: GPT-4, g: LIWC	33.3	71.3	100	0.91
f: GPT-4, g: GPT-4, 0-shot	**39.3**	**77.3**	73	0.68
f: GPT-4, g: GPT-4, 2-shot	**39.3**	**77.3**	–	–
Supervised Models (leave-one-out)				
BERT	35.3	71.3	–	–
GPT-3-emb. + XGBoost	35.3	68.0	–	–
LIWC + XGBoost	34.0	67.3	–	–

Table 1: Accuracy (Acc) and Accuracy@2 (Acc@2) across models. % wins indicate human pairwise evaluation for the explanations with Fleiss' κ (Fleiss and Cohen, 1973) scores as agreement.

5 Experiments

Our experiments are driven by the following hypotheses, and results are analyzed accordingly:

Can few/zero-shot models detect deception? Yes. Table 1 contains the results. The bottleneck model with GPT-4 as both g and f performs the best in accuracy and accuracy@2. GPT-4 as g consistently yields better performance than GPT-3.5 or GPT-3. Both GPT-3.5 and GPT-4 as f are capable of detecting deception better than random. Base models perform worse than bottleneck models irrespective of the choice of the LLM and yield accuracy the same as random. However, the CoT models further decrease accuracy across all LLMs, primarily attributing to the CoT models' attempt to reason the potentially irrelevant or distracting information present in the conversation.

Why bottleneck models are better than base models? Table 2 shows the results of our ablation